L04 — Linear Arrays: Representation and Traversal

Unit: 1 | **Chapter:** Fundamentals of Data Structures & Arrays | **BT Level:** BT4 | **CO:** CO1, CO2

Learning Objectives

- Describe how arrays are stored in memory (row-major and column-major)
- Calculate the address of any element using the base address formula
- Implement array traversal in different patterns
- Analyze the time and space complexity of traversal operations

---

1. What is an Array?

An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**, accessed by an integer **index**.

```
Index:  0    1    2    3    4
Value: [10] [25] [37] [42] [58]
        ↑
    Base address (BA)
```

Key properties:

- Fixed size (declared at creation)
- O(1) random access via index
- Homogeneous elements (same type)
- Zero-indexed (in C, Java, Python) or one-indexed (in Fortran, MATLAB)

---

2. Memory Representation of a 1D Array

Each element occupies a fixed number of bytes determined by its data type:

Type	Size
`int` (C)	4 bytes
`double` (C)	8 bytes
`char` (C)	1 byte

Address Calculation Formula

$$\text{Address}(A[i]) = \text{BA} + i \times \text{size}$$

where:

- **BA** = Base address of array
- **i** = Index of element
- **size** = Size of each element in bytes

Example: BA = 1000, `int` array, find address of A[3]: $$\text{Address}(A[3]) = 1000 + 3 \times 4 = 1012$$

---

3. 2D Array Representation

A 2D array `A[m][n]` has m rows and n columns.

3.1 Row-Major Order (C, C++, Java, Python)

Elements are stored **row by row**:

```
A[0][0], A[0][1], A[0][2], A[1][0], A[1][1], A[1][2], ...
```

Address Formula (0-indexed): $$\text{Address}(A[i][j]) = \text{BA} + (i \times n + j) \times \text{size}$$

Example: BA = 2000, `int` 3×4 array, find A[2][1]: $$= 2000 + (2 \times 4 + 1) \times 4 = 2000 + 9 \times 4 = 2036$$

3.2 Column-Major Order (Fortran, MATLAB)

Elements are stored **column by column**:

```
A[0][0], A[1][0], A[2][0], A[0][1], A[1][1], A[2][1], ...
```

Address Formula (0-indexed): $$\text{Address}(A[i][j]) = \text{BA} + (j \times m + i) \times \text{size}$$

---

4. Array Traversal

Traversal means visiting each element exactly once to read or process it.

4.1 Linear Traversal — O(n)

```
def traverse(arr):
    n = len(arr)
    for i in range(n):
        print(arr[i])     # Process each element
```

Time Complexity: O(n) — visits each of n elements once
Space Complexity: O(1) — no extra memory beyond index variable

4.2 Reverse Traversal — O(n)

```
def traverse_reverse(arr):
    n = len(arr)
```

```
    for i in range(n - 1, -1, -1):
        print(arr[i])
```

4.3 2D Array Traversal — O(m × n)

```
def traverse_2d(matrix, m, n):
    for i in range(m):          # O(m)
        for j in range(n):      # O(n)
            print(matrix[i][j]) # O(1)
# Total: O(m × n)
```

4.4 Traversal Patterns (Interview Common)

```
# Diagonal traversal of a square matrix
def diagonal_traverse(matrix, n):
    for i in range(n):
        print(matrix[i][i])   # O(n)

# Spiral traversal (used in competitive programming)
def spiral_traverse(matrix, m, n):
    top, bottom, left, right = 0, m-1, 0, n-1
    while top <= bottom and left <= right:
        for i in range(left, right + 1):
            print(matrix[top][i])
        top += 1
        for i in range(top, bottom + 1):
            print(matrix[i][right])
        right -= 1
        if top <= bottom:
            for i in range(right, left - 1, -1):
                print(matrix[bottom][i])
            bottom -= 1
        if left <= right:
            for i in range(bottom, top - 1, -1):
                print(matrix[i][left])
            left += 1
```

---

5. Advantages and Disadvantages of Arrays

Advantages

Feature	Detail
Random access	O(1) access via index
Cache-friendly	Contiguous memory → good spatial locality
Simple implementation	No pointer overhead

Disadvantages

Limitation	Detail

Fixed size	Must declare size upfront
Costly insertion/deletion	Shifting elements → O(n)
Wasted space	Pre-allocated size may go unused
Homogeneous	All elements must be same type

---

6. Complexity Summary

Operation	Time Complexity	Space Complexity
Access A[i]	O(1)	O(1)
Traverse all	O(n)	O(1)
2D Traverse	O(m × n)	O(1)
Address calculation	O(1)	O(1)

---

7. Practical Code Example

```
# Complete example: declare, initialize, traverse, and find max
def array_operations():
    arr = [64, 34, 25, 12, 22, 11, 90]
    n = len(arr)

    # Traversal
    print("Array elements:")
    for i in range(n):
        print(f"arr[{i}] = {arr[i]}")

    # Finding maximum during traversal
    max_val = arr[0]
    max_idx = 0
    for i in range(1, n):
        if arr[i] > max_val:
            max_val = arr[i]
            max_idx = i

    print(f"Maximum: {max_val} at index {max_idx}")

array_operations()
```

Output:

```
arr[0] = 64
arr[1] = 34
...
arr[6] = 90
Maximum: 90 at index 6
```

Summary

- Arrays store homogeneous elements in **contiguous memory** — enabling O(1) access.
- **Address formula:** BA + i × size (1D); BA + (i×n + j) × size (2D, row-major).
- **Traversal** visits each element once → O(n) for 1D, O(m×n) for 2D.
- Row-major (C/Java/Python) vs column-major (Fortran/MATLAB) affects traversal efficiency.
- Arrays excel at random access; they struggle with dynamic resizing and mid-array insertion.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.2 (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, Ch. 4.1 (Springer, 2024)